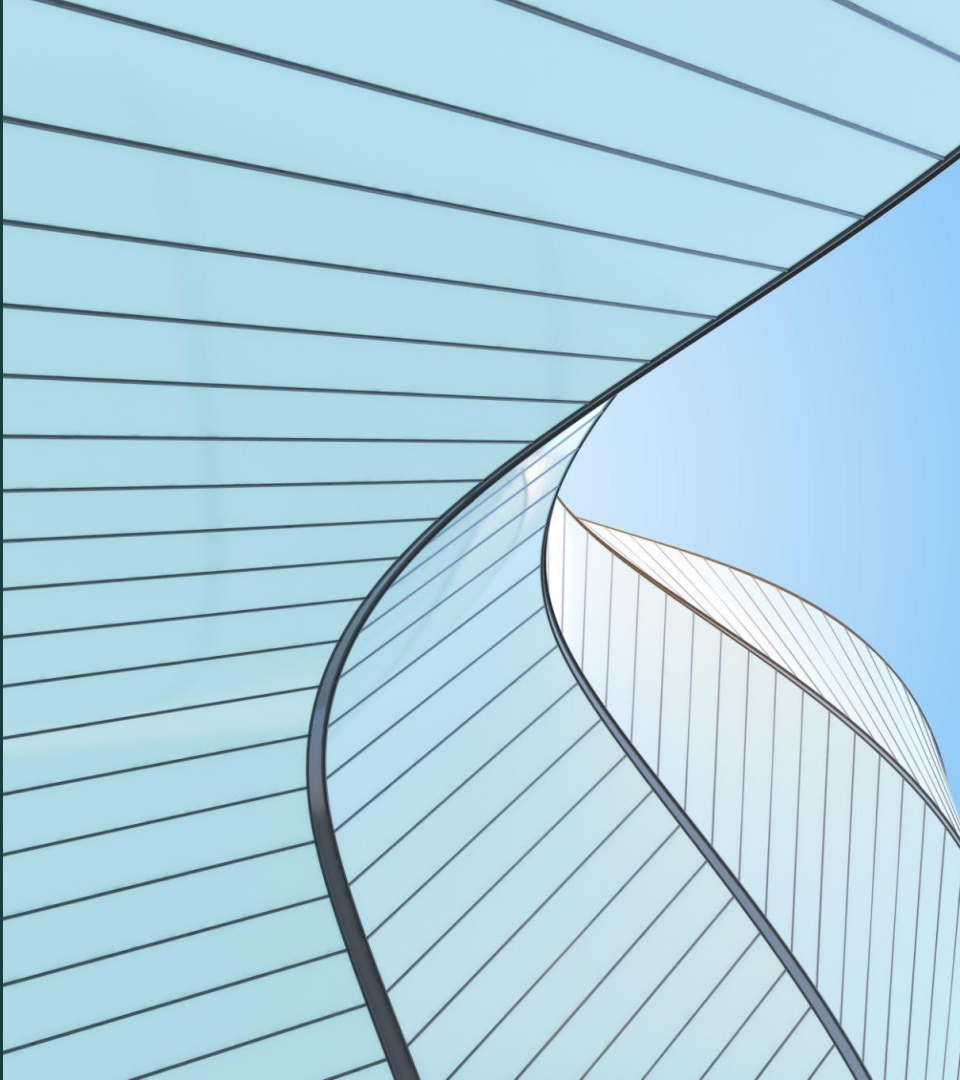


Knowledge Retrieval

*28-day
BOOTCAMP*

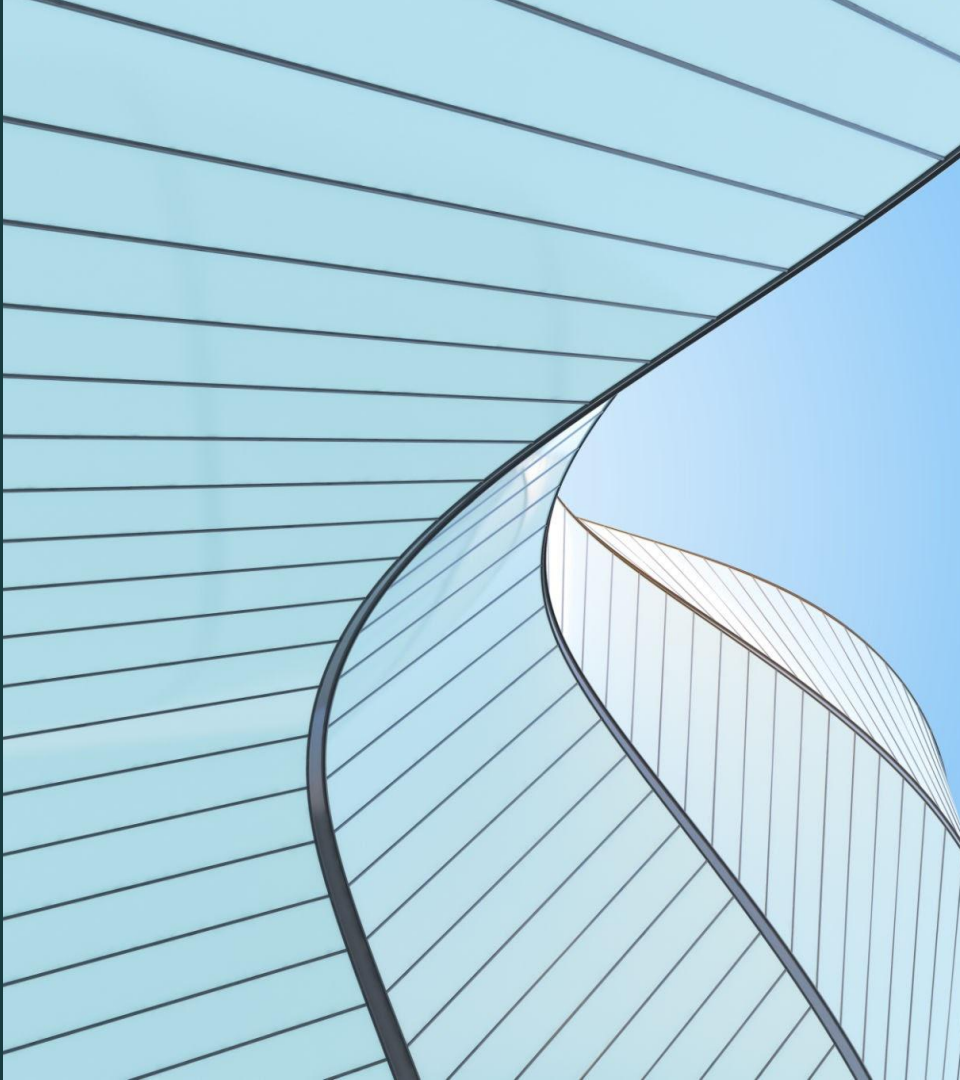
2026

© Sujeet Banerjee



AGENDA

- Knowledge cut off
- Memory $\neq$ Knowledge Base
- *Prompt Optimization (RAG)*
- *RAG - Highlevel Arch.*
- *Exercises, Self-Study*



Recap

- LLM
 - Stateless (no memory)
 - Limited knowledge
- Agent
 - Conv history → Memory
- Challenges
 - Memory poisoning, Split-brain
 - Context Bloating
- User query:
'how many leaves am I entitled to take in a year?'

Bringing in - *Knowledge...*

- “*Knowledge cutoff!*”
 - Does LLMs really learn during *inferencing*?
 - Can tools solve every problem?
 - HR, Taxation, Employee *policies*?
- Policy documents (demo)
 - What’s the increase in Context tokens?
 1. Code_of_Conduct.pdf – 4KB
 2. Employee_Handbook.pdf – 93KB
 3. IT_Security_Policy.pdf – 4KB
 4. Travel_Policy.pdf – 4KB

Context Bloating

PDF	Pages	Words	Estimated Tokens
Code_of_Conduct.pdf	2	696	~900–1,400
Employee_Handbook.pdf	3	902	~1,200
IT_Security_Policy.pdf	2	686	~900–1,250
Travel_Policy.pdf	2	709	~950–1,350

Total pages: 9

Total words: ~3000

Prompt size incr: ~5,000 tokens

Possible Solution - *selective retrieval...*

Retrieve only fragments

Question:

Can I take my company laptop home?

≈180 tokens

⇒ Retrieve paragraph 4

<< Employee Handbook >>

*** Laptop Usage section*

≈160 tokens

⇒ Retrieved paragraph 5

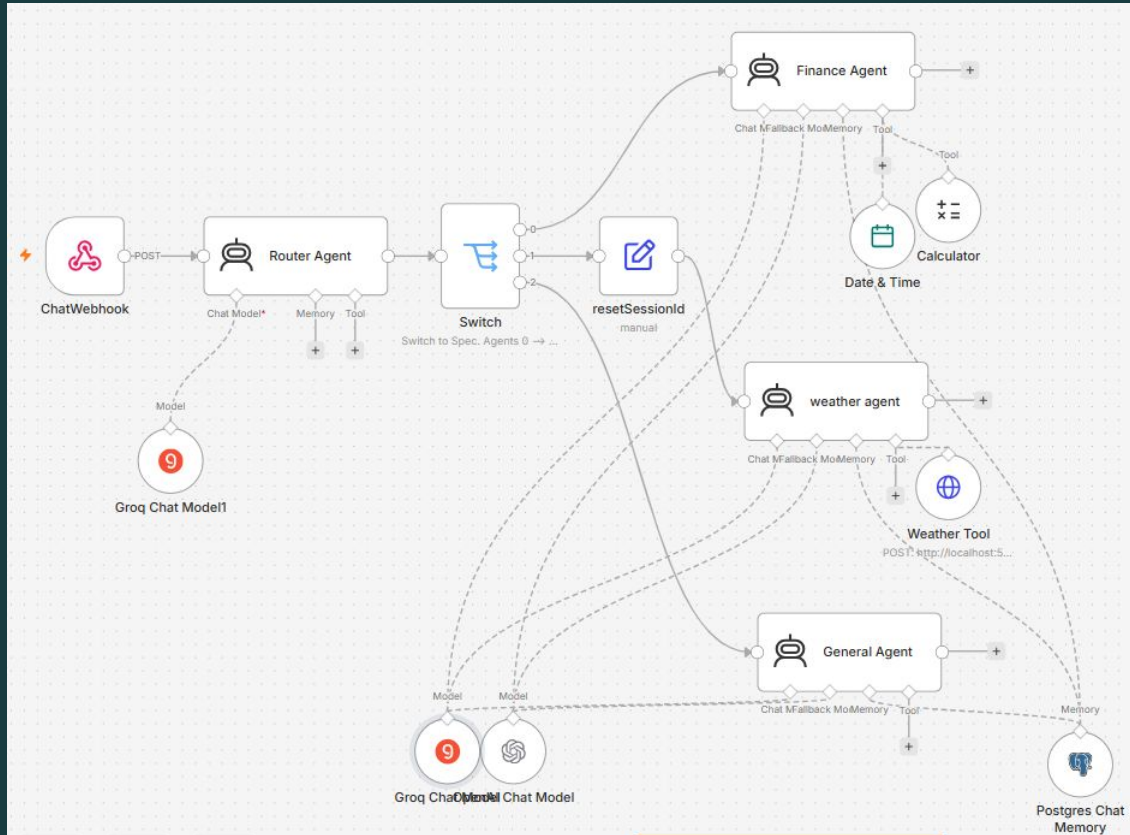
<< IT Security Policy Doc>>

*** Laptop Security section*

Total retrieved context:

≈340 tokens

Where is *knowledge* integrated??



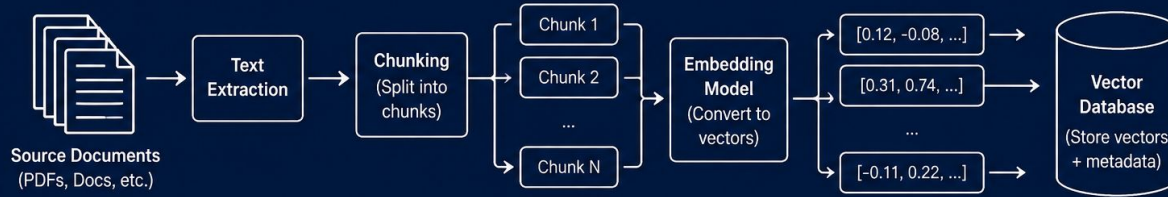
Retrieval Augmented Generation... RAG

OPEN QUESTIONS

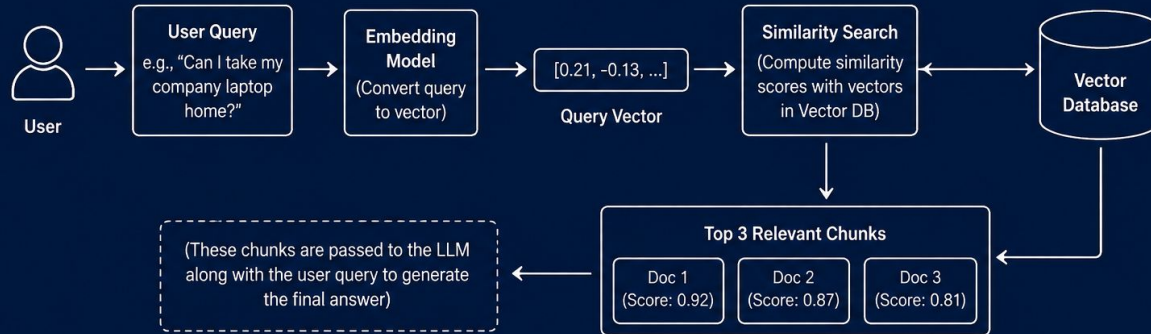
- How do we retrieve only the *relevant paragraphs*?
- What if there are *no paragraphs*?
- What if the sentences are broken/truncated?

RAG - *at a 50' feet height*

PHASE 1: INGESTION (Indexing)



PHASE 2: RETRIEVAL (Query Time)



Summary

- ✓ LLM does not learn during inferencing.
- ✓ Avoid context bloating by bringing only relevant chunks
- ✓ RAG: two phases
 - Ingestion (divide): chunking
 - Retrieval (conquer): bring only the relevant chunks
- ✓ Ensuring only relevant chunks are retrieved is a hard problem to solve. Often non-deterministic

Assignment, Self-study

✓ TBD

*Thank
you...*

BACKUP SLIDES

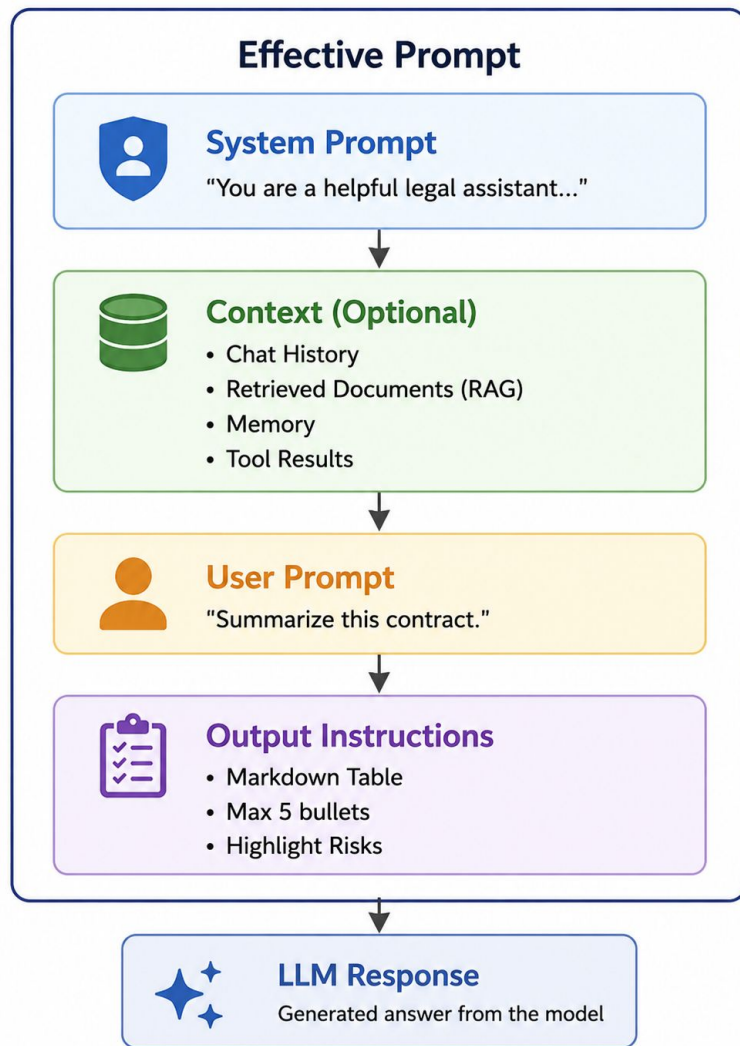
Prompt Arch

Types of prompting

- Zero-shot
- One-shot
- Few-shot
- Role based
- structured

Advanced prompting (coming weeks..)

- CoT, ToT, GoT
- ReAct, Tool-Call, Function-Call
- Reflexive, LLM-as-a-Judge
- Agents, Multi-agents, Supervisor...



Role-based Prompting

Summarize this document.

You are a legal analyst.

Summarize the contract.

Highlight:

- *Risks*
- *Payment terms*
- *Termination*

Return as markdown table.

Prompting - Common mistakes

Too vague

Too broad

No audience

No format

No constraints

Contradictory instructions

Common Strategies:

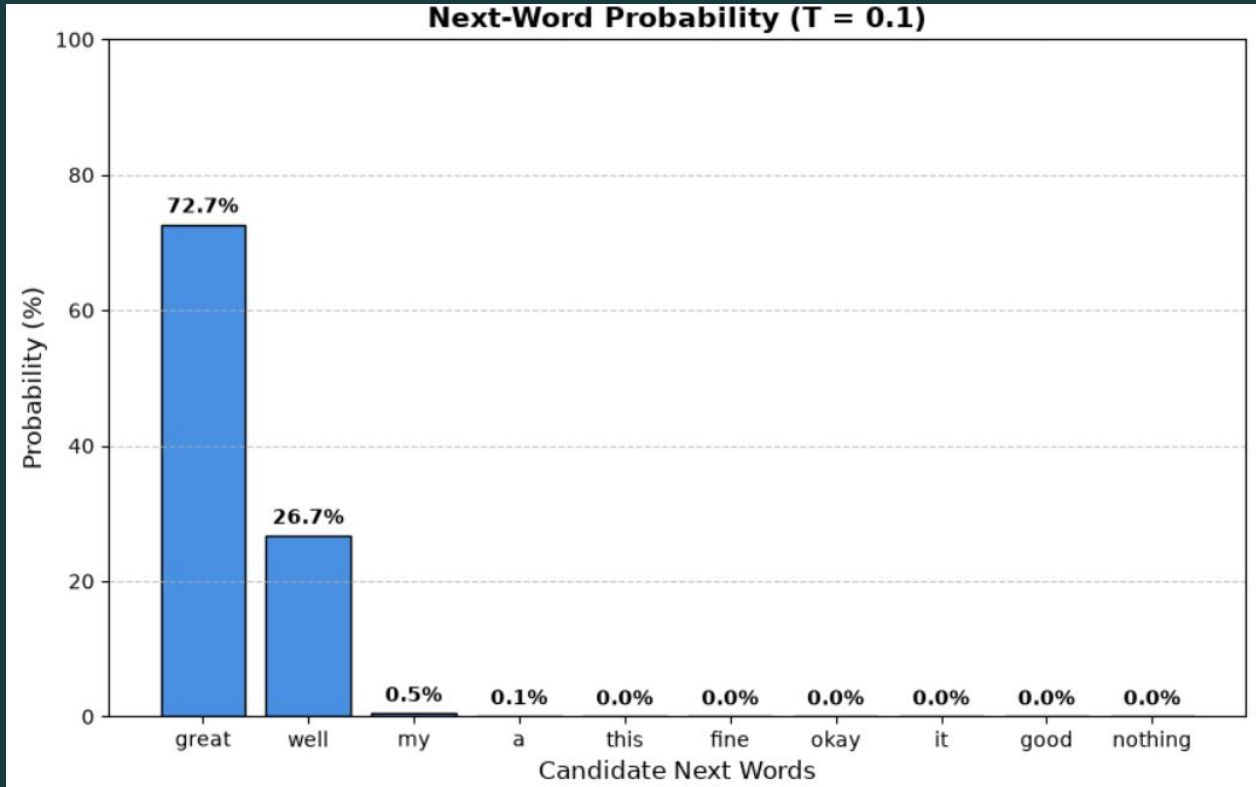
PROMPT-TEMPLATES (week 6)

PROMPT-VERSIONING (week 5)

PROMPT-REDACTION (week 6)

Guess the next, continued...

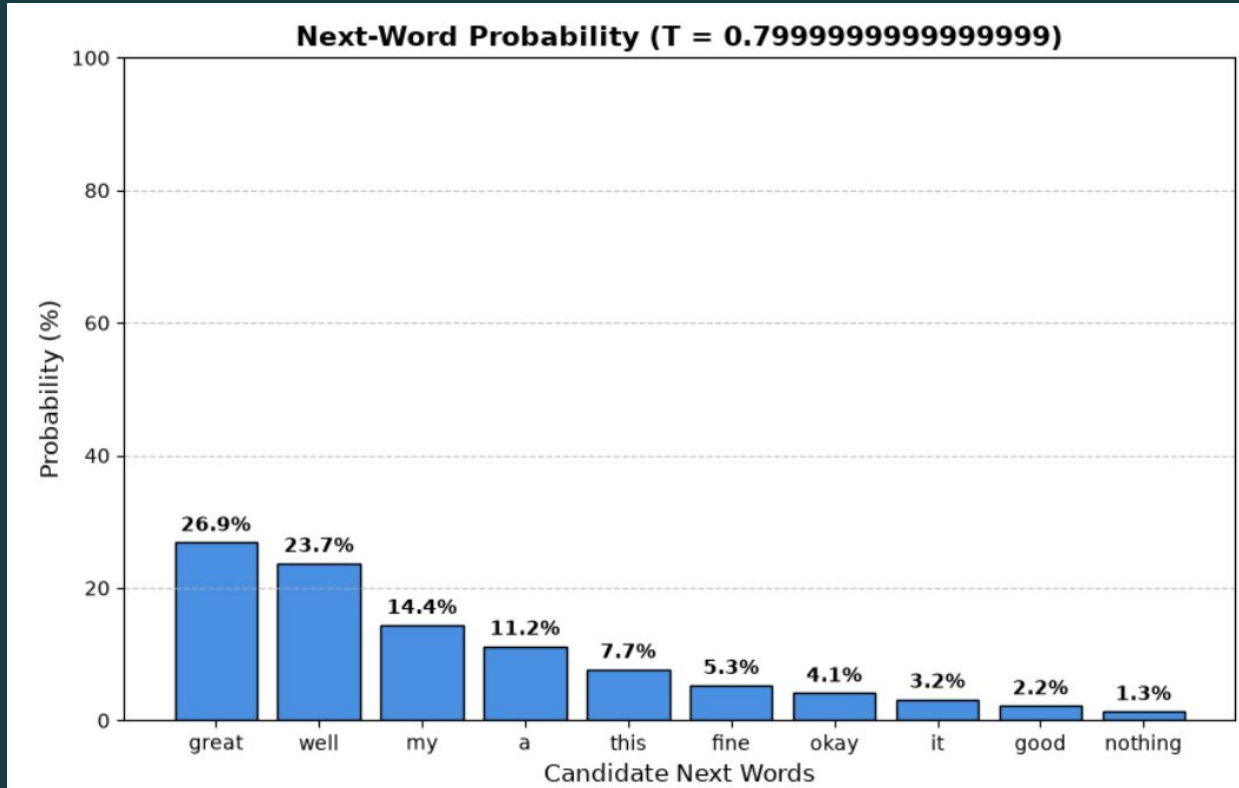
I am doing ?



Temperature = 0.1

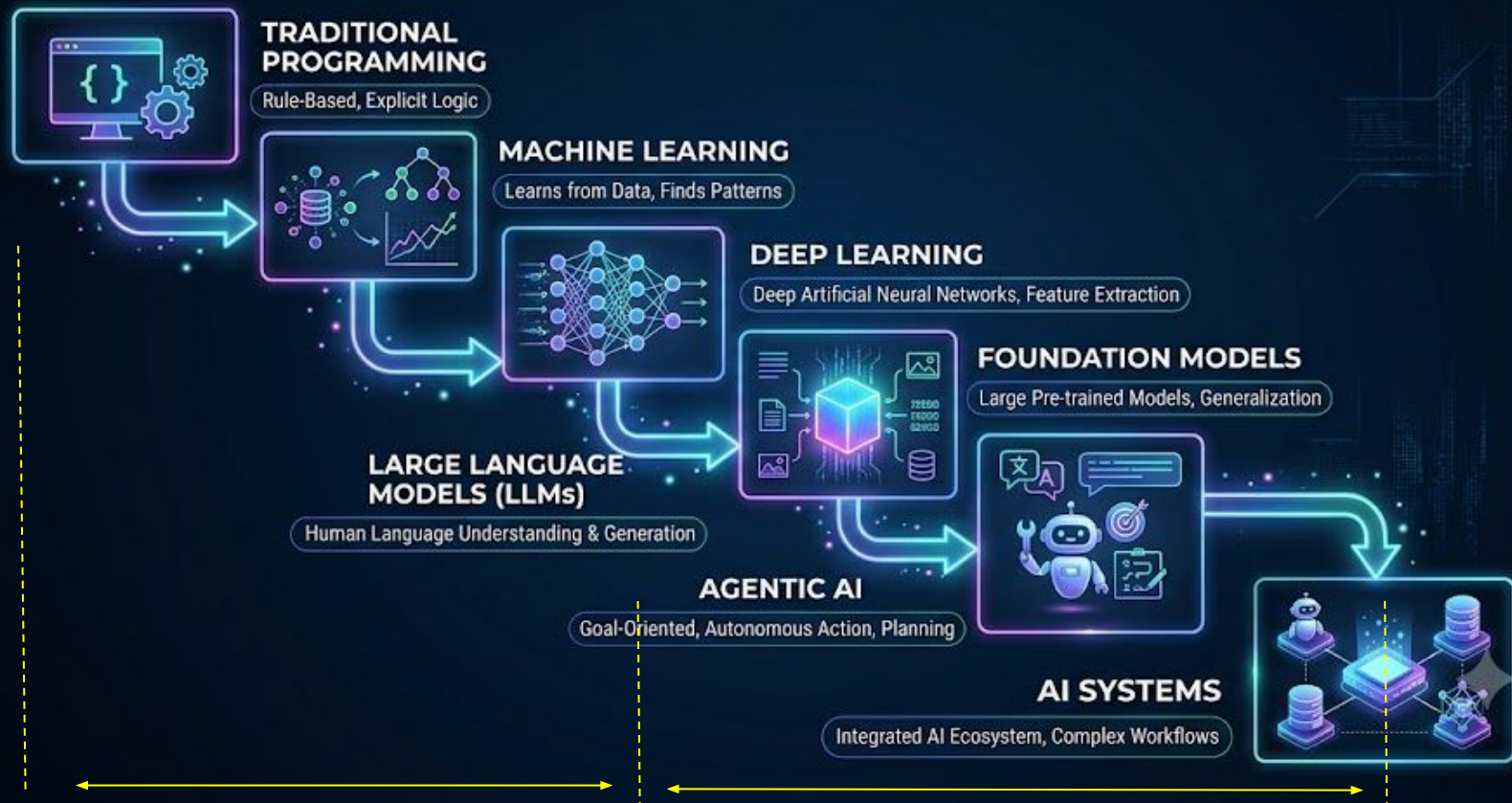
Guess the next, continued...

I am doing ?



Temperature = 0.8

EVOLUTION OF ARTIFICIAL INTELLIGENCE



What are *TOKENS*...?

Why do we care?

- Pricing
- Speed
- Context Window

- Ch-tok
- Word-tok
- Subword
- Byte-Pair-Enc.
- ...

"Artificial Intelligence
is amazing"

['Artificial',
'intelligence', 'is',
'amazing']

"Unbelievable"

['Un', 'believ', 'able']

"Playification"

"Supercalifragilisticexpialidocious"

"こんにちは "

Tokenization

Model	Tokenizer
GPT-2	Byte Pair Encoding (BPE)
GPT-3 / GPT-4 / GPT-5 family	Byte-level BPE (OpenAI variants)
Llama 2 / 3 / 3.2	SentencePiece
Gemma	SentencePiece
Mistral	SentencePiece
Qwen	Byte-level BPE variant
DeepSeek	BPE-based tokenize

Context...

- The number of tokens that for the current input to the LLM – *context*
- The total capacity (in token terms) of input – *context window*
- The model's fundamental unit of computation – *tokens*

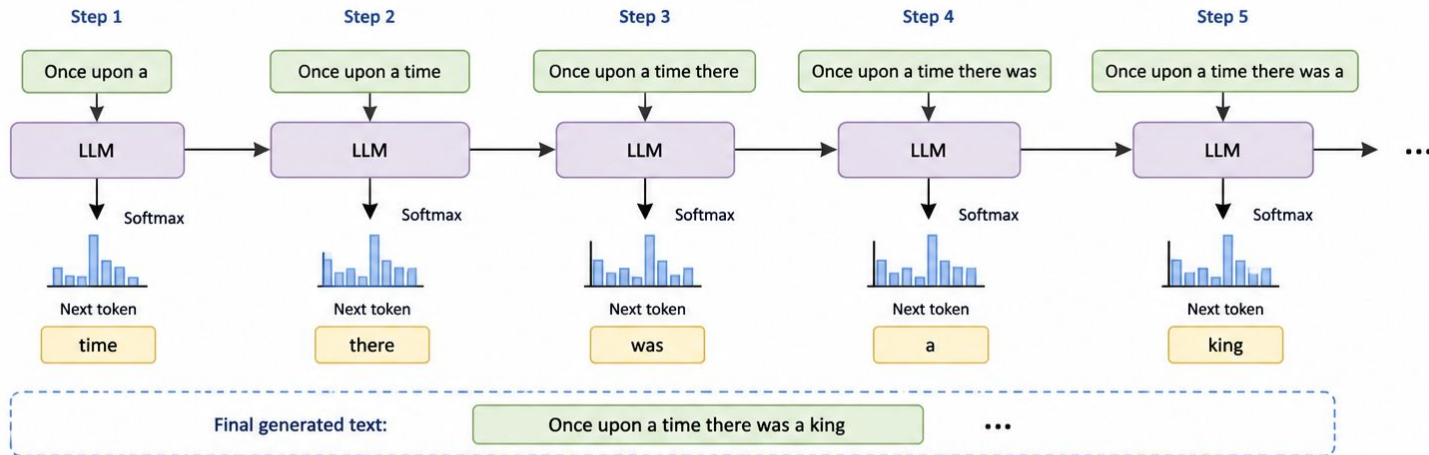
LLM

HIGH LEVEL
ARCH.

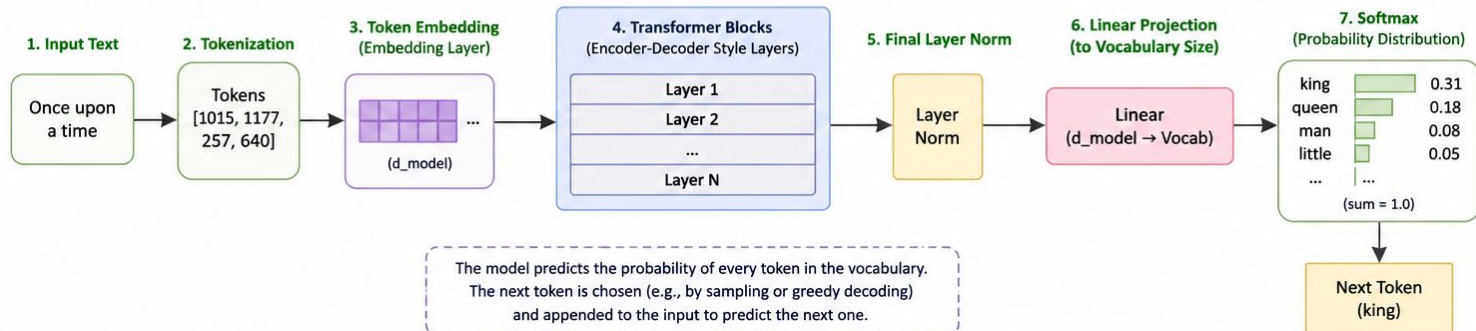
One token
at a time...

AUTOREGRESSIVE GENERATION (NEXT TOKEN PREDICTION)

The model generates one token at a time. Each new token is appended to the context and used to predict the next.



HIGH-LEVEL LLM ARCHITECTURE



WEEK-WISE SCHEDULE

WEEK-1, 2 AI Fundamentals and Evaluation	- LLM fundamentals (evol. From NN / DL) - Prompting architectures - Agentic Foundation (Agentic Arch. patterns) - Metrics: BLEU/ROUGE, Ctx-precision, Recall, Relevance, ... - Intro to RAGAS
WEEK-2, 3 Agentic AI with N8N	- Nodes, Workflows, Triggers, - API Calling, Webhooks, - LLM Integration, Statelessness, Model abstraction - Tool calling, memory (statefulness), Conv. history - Agent Routing, Multi-agent arch.
WEEK-4, 5 RAG Architecture (N8N)	- RAG, EMbedding Fundamentals, - Similarity Search, Vector-DB (Qdrant) - Data-Ingestion, Enrichment, Retrieval Strategies
WEEK- 6 LangFuse, LLMOps	- Telemetry using LiteLLM Proxy - Online and Offline Evals (RAGAS) - LangFuse Dashboarding

INFRA

OPTION 1	OPTION 2
- Local docker-container based setup - Requires Laptop(win) / MacBook - 16GB RAM - 500GB HDD - Installation instructions will be shared for self-hosting. - No guidance provided for setup/installation in the bootcamp	- Hosted on service provide cloud - Use limited-period trial licenses (free period): - N8N - LangFuse - Requires usual laptop - No guidance provided for setup/installation in the bootcamp

AI Licenses

- OpenAI - \$5 token prepaid (optional)
- Groq / Llama3
- Huggingface
- QDrant

LEARNING OUTCOMES

- Understand modern AI architecture
- Build Agentic workflows
- Evaluate AI
- Implement RAG
- Understand LLMOps

Sujeet Banerjee



Welcome



Console



Collections



Tutorial



Datasets



Access Tokens



mini_enterprise_rag



AI EMAIL DRAFTING: CONFLICTING EXPECTATIONS & ACCOUNTABILITY

